

Software Engineering Learning Roadmap

Master Prompt — From Fresh IT Graduate to Job-Ready Software Engineer

Master Prompt — Software Engineering Learning Roadmap & Study Tracker

Your Main Responsibility

IMPORTANT: Do Not Teach Me Randomly

Phase 0 — Programming & Computer Basics

Phase 1 — Programming Fundamentals

Phase 2 — Problem Solving & Computational Thinking

Phase 3 — Data Structures & Algorithms

Phase 4 — Git, GitHub & Developer Tools

Phase 5 — Computer Science Fundamentals

Phase 6 — SQL & Databases

Phase 7 — Web Development Fundamentals

Phase 8 — Backend Development

Phase 9 — Frontend Engineering

Phase 10 — Software Engineering Principles

Phase 11 — Testing & Quality

Phase 12 — System Design

Phase 13 — DevOps & Deployment

Phase 14 — Software Architecture

Phase 15 — Professional Engineering Workflow

Phase 16 — Real-World Projects

Learning Method

Knowledge Checklist

Master Progress Tracker

Learning Resources

Videos

Study Schedule

Weekly Study System

Monthly Assessment

Do Not Let Me Cheat Myself

AI Usage

Career Preparation

Important Rules For You

Most Important Requirement

Final Goal

Master Prompt — Software Engineering Learning Roadmap & Study Tracker

Act as my **personal software engineering mentor, curriculum designer, and study coach.**

About Me

I am a **fresh graduate in Information Technology**.

I have some basic programming knowledge, but I do not consider myself a strong programmer yet. I want to systematically develop myself into a **professional Software Engineer**, not just someone who knows how to use programming frameworks.

My goal is to understand:

- How computers and software fundamentally work
- How to think like a programmer
- How to solve problems systematically
- How to write clean and maintainable code
- How software is designed and architected
- How databases work
- How APIs and distributed systems work
- How applications communicate over networks
- How to test and debug software
- How Git and professional development workflows work
- How software is deployed and maintained
- How to work on real-world software projects
- How professional software engineers think and make technical decisions
- How to prepare for software engineering jobs

I want to learn **proper fundamentals first**, rather than jumping immediately between frameworks and technologies.

Your Main Responsibility

Create a complete, structured, long-term **Software Engineering Learning Roadmap** for me.

Treat this like designing a university-level curriculum combined with practical industry training.

The roadmap should take me from:

Beginner → Strong Programmer → Junior Software Engineer → Job-Ready Software Engineer

Do NOT assume that I already understand computer science fundamentals.

If something is important, teach it from the beginning.

IMPORTANT: Do Not Teach Me Randomly

Organize everything into logical phases.

Each phase should build upon the previous phase.

For every phase, provide:

1. What I need to learn
2. Why I need to learn it
3. Concepts I need to understand
4. Practical exercises

5. Programming exercises
6. Mini-projects
7. One larger project when appropriate
8. Recommended videos
9. Recommended websites
10. Recommended documentation
11. Recommended books/courses when useful
12. A checklist
13. A test/self-assessment
14. A definition of what “finished” means

Do not simply give me a list of technologies.

Explain the reasoning behind the learning order.

Phase 0 — Programming & Computer Basics

Start by evaluating whether I understand the basic concepts.

Cover:

- What a computer actually does
- CPU
- RAM
- Storage
- Operating systems
- Processes
- Files
- File systems
- Programs
- Compilers
- Interpreters
- Runtime
- Source code
- Machine code
- Memory
- Stack vs heap
- Environment variables
- Command line
- Terminal
- Paths
- Packages/dependencies
- How programs are executed

Teach these concepts in beginner-friendly language.

Give me small experiments that I can perform on my own computer.

Phase 1 — Programming Fundamentals

Teach programming fundamentals deeply.

Topics should include:

- Variables
- Data types
- Operators
- Input/output
- Conditional statements
- Loops
- Functions
- Parameters
- Return values
- Scope
- Collections
- Strings
- Error handling
- Modules
- Packages
- File handling
- Basic debugging
- Basic object-oriented programming
- Classes
- Objects
- Encapsulation
- Inheritance
- Polymorphism
- Composition

Do not allow me to simply memorize syntax.

For every major concept:

Explain:

Concept → Why it exists → Example → Exercise → Real-world use

Give me progressively harder exercises.

Phase 2 — Problem Solving & Computational Thinking

Teach me how software engineers actually approach problems.

Cover:

- Breaking large problems into smaller problems
- Decomposition
- Abstraction
- Pattern recognition
- Algorithms
- Pseudocode
- Flowcharts
- Time complexity
- Space complexity

- Big-O notation
- Recursion
- Searching
- Sorting
- Two pointers
- Hashing
- Basic problem-solving patterns

Give me programming problems.

For each problem, teach me to think through:

1. What is the problem?
2. What are the inputs?
3. What are the outputs?
4. What constraints exist?
5. What is the simplest solution?
6. Can it be optimized?
7. What is the time complexity?
8. What is the space complexity?
9. What edge cases exist?

Do not immediately show the solution.

Let me attempt problems first.

Phase 3 — Data Structures & Algorithms

Teach data structures from fundamentals.

Include:

- Arrays
- Strings
- Linked lists
- Stacks
- Queues
- Hash tables
- Sets
- Trees
- Binary trees
- Binary search trees
- Heaps
- Graphs
- Tries when appropriate

Then algorithms:

- Linear search
- Binary search
- Sorting algorithms
- Tree traversal
- Graph traversal

- BFS
- DFS
- Recursion
- Dynamic programming fundamentals
- Greedy algorithms
- Common algorithmic patterns

For each data structure:

Explain:

- What problem it solves
- How it works internally
- Its strengths
- Its weaknesses
- Time complexity
- Space complexity
- When to use it
- When NOT to use it
- Real-world examples

Require me to implement important structures myself.

Phase 4 — Git, GitHub & Developer Tools

Teach me professional development tools.

Cover:

- Terminal
- Linux basics
- Shell commands
- Git
- GitHub
- Branches
- Commits
- Merge
- Rebase
- Pull requests
- Merge conflicts
- .gitignore
- Tags
- Releases
- Git history
- Git workflows
- SSH
- Environment variables
- Package managers
- Debuggers
- IDEs
- Code formatting
- Linters

Teach Git practically.

Give me scenarios such as:

- I accidentally committed something
- My branch is behind
- GitHub rejected my push
- I have local changes and need to pull
- I need to undo a commit
- I need to resolve a merge conflict
- I need to work on a feature branch
- I need to review someone else's pull request

Do not only teach commands. Explain what Git is doing internally.

Phase 5 — Computer Science Fundamentals

Teach the core CS concepts that a software engineer should understand.

Include:

Operating Systems

- Processes
- Threads
- Concurrency
- Memory
- Virtual memory
- Scheduling
- Filesystems
- Permissions
- System calls

Networking

- IP addresses
- TCP
- UDP
- DNS
- HTTP
- HTTPS
- Ports
- Sockets
- Cookies
- Sessions
- TLS
- Proxies
- Reverse proxies

Databases

- Relational databases
- Tables
- Rows
- Columns
- Primary keys
- Foreign keys
- Relationships
- Normalization
- Indexes
- Transactions
- ACID
- Constraints
- Joins
- Query optimization

Security Fundamentals

- Authentication
- Authorization
- Sessions
- Password hashing
- Encryption
- HTTPS
- SQL injection
- XSS
- CSRF
- CORS
- Secrets management
- Principle of least privilege

Explain the concepts conceptually before teaching implementation.

Phase 6 — SQL & Databases

Teach SQL deeply.

Start with:

- SELECT
- INSERT
- UPDATE
- DELETE
- WHERE
- ORDER BY
- GROUP BY
- HAVING
- JOIN
- Subqueries
- Aggregation
- Constraints

- Indexes
- Transactions
- Views
- Database design

Then teach me how applications communicate with databases.

Teach:

Application → ORM/Database Driver → SQL → Database

Explain what actually happens.

Give me database design exercises.

For example:

- Inventory system
- E-commerce system
- Employee management system
- School management system
- Payroll system

Require me to design the database before writing application code.

Phase 7 — Web Development Fundamentals

Teach web development from the fundamentals.

Cover:

- HTML
- CSS
- JavaScript
- Browser
- DOM
- HTTP
- REST
- JSON
- Forms
- Cookies
- Sessions
- Authentication
- Authorization
- APIs
- Client/server architecture

Explain:

Browser → HTTP Request → Web Server → Application → Database → Response → Browser

Make sure I understand what happens at every step.

Do not let me rely on frameworks without understanding what they are abstracting.

Phase 8 — Backend Development

Teach backend engineering.

Cover:

- Routing
- Controllers
- Services
- Business logic
- Models
- Repositories
- Middleware
- Authentication
- Authorization
- Validation
- Error handling
- Logging
- Configuration
- Environment variables
- Database access
- API design
- REST APIs
- Pagination
- Filtering
- Sorting
- File uploads
- Background jobs
- Caching

Teach me how to structure a maintainable backend.

Explain concepts such as:

- Separation of concerns
- Dependency injection
- Single responsibility
- Layered architecture
- Service layer
- Repository pattern
- DTOs
- Domain logic

Phase 9 — Frontend Engineering

Teach modern frontend development after I understand web fundamentals.

Cover:

- JavaScript deeply
- DOM
- Async programming
- Promises
- Fetch

- Modules
- State
- Components
- Forms
- API integration
- Error handling
- Accessibility
- Responsive design
- Frontend architecture

Only introduce frontend frameworks after I understand the underlying concepts.

Explain what frameworks are solving.

Phase 10 — Software Engineering Principles

Now teach me how to move from “programmer” to “software engineer.”

Cover:

- Clean code
- SOLID
- DRY
- KISS
- YAGNI
- Separation of concerns
- Coupling
- Cohesion
- Refactoring
- Technical debt
- Code smells
- Design patterns
- Dependency inversion
- Abstraction
- Maintainability
- Readability
- Scalability

Teach these through real code examples.

Show me:

Bad code → Problems → Refactoring → Better code

Do not teach design patterns as memorization.

Teach me when NOT to use them.

Phase 11 — Testing & Quality

Teach professional software testing.

Cover:

- Why testing matters
- Unit testing
- Integration testing
- End-to-end testing
- Test cases
- Assertions
- Mocking
- Fixtures
- Test-driven development
- Regression testing
- Edge cases
- Test coverage

Give me projects where I must write tests.

Teach me how to debug:

Bug → Reproduce → Isolate → Understand → Fix → Test → Prevent regression

Phase 12 — System Design

After I understand programming, databases, networking and backend development, introduce system design.

Start simple.

Teach:

- Requirements
- Functional requirements
- Non-functional requirements
- Scalability
- Availability
- Reliability
- Latency
- Throughput
- Load balancing
- Caching
- Queues
- Databases
- Replication
- Partitioning
- Horizontal scaling
- Vertical scaling
- Microservices
- Monoliths
- Event-driven systems

Start with small systems.

Examples:

- URL shortener
- Authentication system

- File storage system
- Chat application
- Notification system
- E-commerce system

Always explain the trade-offs.

Do not teach system design as drawing boxes.

Teach me how to make engineering decisions.

Phase 13 — DevOps & Deployment

Teach me how software gets from my computer to real users.

Cover:

- Linux servers
- Processes
- Environment variables
- Web servers
- Reverse proxies
- Domains
- DNS
- HTTPS
- Docker
- Containers
- CI/CD
- GitHub Actions
- Logging
- Monitoring
- Backups
- Deployment strategies
- Cloud fundamentals

Eventually make me deploy real applications.

Phase 14 — Software Architecture

Teach architecture after I have enough practical experience.

Cover:

- Monolithic architecture
- Modular monolith
- Layered architecture
- Clean architecture
- Hexagonal architecture
- Microservices
- Event-driven architecture
- Domain-driven design fundamentals
- API architecture

- Service boundaries
- Dependency management

Emphasize:

Architecture is about trade-offs, not fashionable patterns.

Phase 15 — Professional Engineering Workflow

Teach me how professional software teams work.

Cover:

- Requirements
- User stories
- Acceptance criteria
- Issue tracking
- Git workflow
- Branching strategies
- Pull requests
- Code review
- Documentation
- Testing
- CI/CD
- Releases
- Versioning
- Bug tracking
- Incident management
- Technical debt
- Agile/Scrum fundamentals

Give me simulated team scenarios.

For example:

“You are assigned a ticket. What do you do?”

Make me think through the process instead of just coding.

Phase 16 — Real-World Projects

I want projects that progressively become more difficult.

Create projects in levels:

Level 1 — Beginner

Examples:

- CLI calculator
- To-do application
- File organizer
- Contact manager

Level 2 — Intermediate

Examples:

- Inventory system
- POS system
- Expense tracker
- Authentication system
- REST API

Level 3 — Advanced

Examples:

- Multi-user SaaS application
- E-commerce backend
- Project management system
- Payroll/workforce system
- Real-time application

Level 4 — Software Engineering Project

Create one serious portfolio project that requires:

- Authentication
- Authorization
- Database
- API
- Frontend
- Testing
- Logging
- Documentation
- Git workflow
- CI/CD
- Deployment
- Security considerations
- Architecture documentation

For each project, provide:

- Requirements
- Features
- Database design
- Architecture
- Technology choices
- Development milestones
- Git workflow
- Testing requirements
- Deployment requirements
- README requirements

Do not give me the entire solution immediately.

Make me build it.

Learning Method

I do NOT want to passively watch tutorials.

For every topic use approximately:

20% learning + 60% building/practice + 20% review

Use active learning.

After teaching a topic:

1. Ask me questions.
2. Give me a small exercise.
3. Give me a coding problem.
4. Ask me to explain the concept in my own words.
5. Give me a debugging scenario.
6. Give me a practical task.
7. Review my answer.
8. Identify my weaknesses.
9. Tell me what I should review.

Knowledge Checklist

Create a checklist for every phase.

Use this format:

Phase X — Topic

- ☐ I understand the concept
- ☐ I can explain it without notes
- ☐ I can implement it
- ☐ I can debug it
- ☐ I can explain when to use it
- ☐ I can explain when NOT to use it
- ☐ I completed the exercises
- ☐ I completed the project
- ☐ I can apply it in a real application

Do NOT mark anything as completed for me.

I will mark the checklist myself.

Master Progress Tracker

At the end of the roadmap, create a large progress tracker:

Area	Beginner	Learning	Practicing	Competent	Strong
Programming					

Problem solving					
Data structures					
Algorithms					
Git					
Linux					
SQL					
Databases					
Networking					
Web development					
Backend					
Frontend					
APIs					
Security					
Testing					
Debugging					
Software architecture					
System design					
DevOps					
Cloud					
Documentation					
Code review					
Communication					
Professional workflow					

Learning Resources

For every major topic, recommend high-quality resources.

Prioritize:

1. Official documentation
2. University courses
3. High-quality free courses
4. Well-established technical websites
5. Books
6. YouTube lectures

Whenever possible, provide:

- Resource name
- URL
- What it teaches
- Why you recommend it
- Which phase I should use it in

- Whether it is beginner/intermediate/advanced

Prefer authoritative resources over random tutorial websites.

For example, consider resources such as:

- Harvard CS50
- MIT OpenCourseWare
- MIT Missing Semester
- Official Git documentation
- Official language documentation
- Official framework documentation
- MDN
- PostgreSQL documentation
- OWASP
- freeCodeCamp

Do not recommend 20 resources for the same topic.

Give me a **primary resource** and only a few alternatives.

Videos

When a video/course would significantly help, provide a video.

For every recommended video:

- Title
- Creator/institution
- Link
- Approximate length
- What I should learn from it
- What I should do after watching it

Do not give me videos just because they are popular.

Only recommend videos that actually contribute to the curriculum.

Study Schedule

Create several options:

1-hour/day

A sustainable schedule for someone with limited time.

2-hours/day

A serious learning schedule.

4-hours/day

An intensive schedule.

For each schedule, explain:

- Study
- Coding
- Projects
- Review
- Problem solving

Do not encourage burnout.

Weekly Study System

Give me a repeatable weekly structure.

Example:

Monday

- Learn new concept
- Practice

Tuesday

- Exercises
- Coding

Wednesday

- New concept
- Implementation

Thursday

- Problem solving

Friday

- Project

Saturday

- Project + review

Sunday

- Review + assessment

Adapt this structure depending on the phase.

Monthly Assessment

At the end of each major section, create an assessment.

The assessment should test:

Knowledge

Can I explain it?

Implementation

Can I build it?

Problem solving

Can I use it to solve a new problem?

Debugging

Can I identify why something failed?

Engineering judgment

Can I choose the appropriate approach?

Communication

Can I explain my technical decision?

Do Not Let Me Cheat Myself

If I say:

“I watched the tutorial.”

Do NOT consider the topic completed.

If I cannot implement or explain the concept, tell me:

“You have been exposed to this topic, but you have not mastered it yet.”

If I copy code from ChatGPT, Stack Overflow, GitHub, or documentation, encourage me to understand what the code does.

Teach me to use AI as a **software engineering assistant**, not as a replacement for understanding.

AI Usage

Teach me how to use AI effectively as a modern software engineer.

Cover:

- Asking AI technical questions
- Debugging with AI
- Code review with AI
- Understanding generated code
- Writing tests with AI
- Documentation with AI
- Research with AI
- AI-assisted development
- Verifying AI answers
- Detecting hallucinations
- Security risks of AI-generated code

- When NOT to use AI

I should still be able to solve fundamental problems without AI.

Career Preparation

Once my fundamentals are strong, prepare me for actual software engineering employment.

Cover:

- Building a portfolio
- GitHub
- README files
- Technical documentation
- Project demonstrations
- Resume
- LinkedIn
- Technical interviews
- Coding interviews
- Behavioral interviews
- System design interviews
- Code review exercises
- Take-home assignments
- Junior developer expectations

Give me realistic interview questions and practical assessments.

Important Rules For You

1. Do not overwhelm me by teaching everything at once.
2. Give me a clear sequence.
3. Explain why each topic matters.
4. Assume I may have forgotten fundamentals.
5. Prefer understanding over memorization.
6. Prefer building over watching.
7. Prefer official documentation and university-quality material.
8. Do not make me learn multiple programming languages unnecessarily.
9. Do not make frameworks the foundation of my education.
10. Teach transferable concepts.
11. Revisit difficult concepts when necessary.
12. Connect theory to real-world software.
13. Make projects progressively harder.
14. Include debugging.
15. Include testing.
16. Include security.
17. Include Git and professional workflows.
18. Include architecture and system design later.
19. Teach trade-offs rather than presenting one technology as universally correct.
20. Keep the roadmap realistic for a fresh IT graduate.

Most Important Requirement

I want this to become an **interactive study program**, not just a document.

After you create the roadmap:

First

Give me the complete roadmap.

Then

Tell me exactly where I should start.

Then

Give me **Day 1**.

Do NOT give me Day 1 through Day 100 immediately.

Teach me progressively.

When I finish a lesson, I will tell you something like:

Completed Phase 1 — Variables and Data Types.

You should then:

1. Test my understanding.
2. Give me exercises.
3. Review my answers.
4. Identify weaknesses.
5. Decide whether I should continue.
6. Update my recommended next step.

Treat my learning like an ongoing mentorship program.

Final Goal

I don't want to become someone who can only follow tutorials.

I want to become someone who can receive a problem such as:

"Build a system that allows multiple companies to manage employees, roles, payroll data, and reports."

and confidently think:

Requirements → Design → Architecture → Database → Backend → API → Frontend → Security → Testing → Deployment → Monitoring → Maintenance

I want to understand **why** each decision is made, not just know which framework command to type.

Build my curriculum around that goal.